

[Sign in](#)[Contributor I](#)

The world's leading publication for data science, AI, and ML professionals.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[CONTRIBUTE](#)[NEWSLETTER](#)

MACHINE LEARNING

Reinforcement Learning – Implement Grid World From Scratch

When you try to get your hands on reinforcement learning, it's likely that Grid World Game is the very first problem you meet with. It is...

Jeremy Zhang

May 4, 2019 5 min read



Introduction of Value Iteration

When you try to get your hands on [Reinforcement Learning](#), it's likely that **Grid World Game** is the very first problem you meet with. It is the most basic as well as classic problem in reinforcement learning and by implementing it on your own, I believe, is the best way to understand the basis of reinforcement learning. Meanwhile, it is super fun to implement your own game and see how a robot manage to learn on its own!



Rules

			end +1
			end -1
start			

Grid Board

The rule is simple. Your agent/robot starts at the left-bottom corner(the 'start' sign) and ends at either +1 or -1 which is the corresponding reward. At each step, the agent has 4 possible actions including up, down, left and right, whereas the black block is a wall where your agent won't be able to penetrate through. In order to make it more straight forward, our first implementation **assumes that each action is deterministic**, that is, the agent will go where it intends to go. For instance, when the agent decides to take action up at (2, 0), it will land in (1, 0) rather than (2, 1) or elsewhere. (We will add uncertainty in our second implementation) However, if the agents hit the wall, it will remain at the same position.

Board

So let's get cracking on the code! Firstly, let's set up some global rules of the board. ([full code](#))

```
1 # global variables
2 BOARD_ROWS = 3
3 BOARD_COLS = 4
4 WIN_STATE = (0, 3)
5 LOSE_STATE = (1, 3)
6 START = (2, 0)
7 DETERMINISTIC = True
```

gridworld1.py hosted with ♥ by GitHub

[view raw](#)

And as a grid game, it needs a State to justify each state(position) of our agent, giving reward according to its state.

```
1 def giveReward(self):
```

```

2         if self.state == WIN_STATE:
3             return 1
4         elif self.state == LOSE_STATE:
5             return -1
6         else:
7             return 0

```

gridworld2.py hosted with ♥ by GitHub

[view raw](#)

When our agent takes an action, the State should have a function to accept an action and return a legal position of next state.

```

1  def nxtPosition(self, action):
2      """
3      action: up, down, left, right
4      -----
5      0 | 1 | 2 | 3 |
6      1 |
7      2 |
8      return next position
9      """
10
11     if self.determine:
12         if action == "up":
13             nextState = (self.state[0] - 1, self.state[1])
14         elif action == "down":
15             nextState = (self.state[0] + 1, self.state[1])
16         elif action == "left":
17             nextState = (self.state[0], self.state[1] - 1)
18         else:
19             nextState = (self.state[0], self.state[1] + 1)
20         # if next state legal
21         if (nextState[0] >= 0) and (nextState[0] <= 2):
22             if (nextState[1] >= 0) and (nextState[1] <= 3):
23                 if nextState != (1, 1):
24                     return nextState
25         return self.state

```

gridworld3.py hosted with ♥ by GitHub

[view raw](#)

Agent

This is the artificial intelligence part, as our agent should be able to learn from the process and thinks like a human. The key of the magic is value iteration.

Value Iteration

What our agent will finally learn is a policy, and **a policy is a mapping from state to action**, simply instructs what the agent should do at each state. In our case, instead of learning a mapping from state to action, we will leverage value iteration to firstly learn a mapping of state to value(which is the estimated reward) and based on the estimation, at each state, our agent will choose the best action that gives the highest estimated reward.

There is not going to be any cranky, head-scratching math involved, as the core of value iteration is amazingly concise.

$$V(S_t) \leftarrow V(S_t) + \alpha [V(S_{t+1}) - V(S_t)]$$

value iteration

This is the essence of value iteration, super neat, right? This formula almost applies to all reinforcement learning problems, let me explain how our agent evolves from an infant to expert based on this line of formula. Value iteration, just as its name, update its value(estimated reward) at each iteration(end of game).

At first, our agent knows nothing about the grid world(environment), so **it would simply initialise all reward as 0**. Then, it starts to explore the world by randomly walking around, surely it will endure lots of failure at the beginning, but that is totally fine. Once it reaches end of the game, either reward +1 or reward -1, the whole game reset and the **reward propagates in a backward fashion and eventually the estimated value of all states along the way will be updated based on the formula above**.

Let's take a closer look at the formula. **The $V(S_t)$ on the left is the updated value of that state, and the right one is the current non-updated value and α is learning rate**. The formula is simply saying that the updated value of a state equals to the current value plus a temporal difference, which is what the agent learned from this iteration of game playing minus the previous estimate. For example, let's say there are 2 states, S_1 and S_2 , both of which has an estimated value 0, and at this round of playing, our agent moves from S_1 to S_2 and gets reward 1, then the new estimate of $S_1 = S_1 + \alpha(S_2 - S_1)$, which is $0 + 0.1(1-0) = 0.1$ (assume α is 0.1 and reward at S_1 is 0)

```

1  def play(self, rounds=10):
2      i = 0
3      while i < rounds:
4          # to the end of game back propagate reward
5          if self.State.isEnd:
6              # back propagate
7              reward = self.State.giveReward()
8              # explicitly assign end state to reward values
9              self.state_values[self.State.state] = reward # this is optional
10             print("Game End Reward", reward)
11             for s in reversed(self.states):
12                 reward = self.state_values[s] + self.lr * (reward - self.state_val
13                 self.state_values[s] = round(reward, 3)
14             self.reset()
15             i += 1
16         else:
17             action = self.chooseAction()
18             # append trace
19             self.states.append(self.State.nextPosition(action))
20             print("current position {} action {}".format(self.State.state, action))

```

```

21         # by taking the action, it reaches the next state
22         self.State = self.takeAction(action)
23         # mark is end
24         self.State.isEndFunc()
25         print("nxt state", self.State.state)
26         print("-----")

```

gridworld4.py hosted with ♥ by GitHub

[view raw](#)

We record all states of our agent, and at the end of the game, we will update estimates in reversed fashion.

Exploration & Exploitation

There is one last thing we need to talk about. Once our agent finds a path to get reward +1, should it sticks to it and forever follows that path (exploitation) or should it gives other path a chance(exploration) and expects a shorter path? In actual, we will balance exploration and exploitation in order to avoid our agent stuck in local optimal. Here our agent will choose action based on certain exploration_rate

```

1  def chooseAction(self):
2      # choose action with most expected value
3      mx_nxt_reward = 0
4      action = ""
5
6      if np.random.uniform(0, 1) <= self.exp_rate:
7          action = np.random.choice(self.actions)
8      else:
9          # greedy action
10         for a in self.actions:
11             # if the action is deterministic
12             nxt_reward = self.state_values[self.State.nxtPosition(a)]
13             if nxt_reward >= mx_nxt_reward:
14                 action = a
15                 mx_nxt_reward = nxt_reward
16         return action

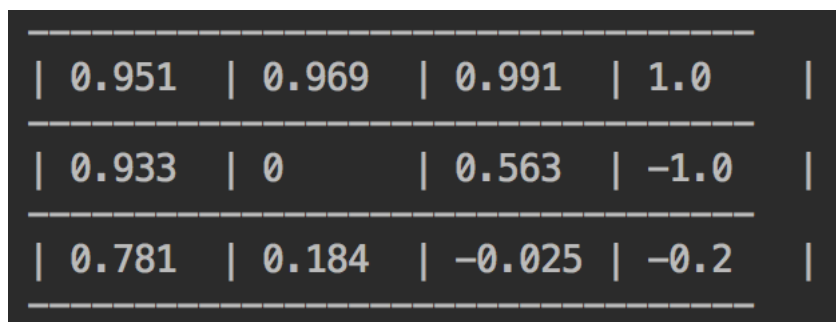
```

gridworld5.py hosted with ♥ by GitHub

[view raw](#)

Play

That's it! These are all we need for a grid world game. We can start and let our agent play the game!



50 round of playing

This is the estimates of each state after playing 50 rounds of game. As our action is deterministic, we can get best action at each state by following the highest estimate! Full code is [here](#), go play with it and welcome to contribute if you find any to update!

For now, we are all focusing on value iteration and deterministic game world. However, in real situation the agent will not always lands in a position where it hopes to go. Let us explore deeper on [non-deterministic game playing and Q-learning](#).

References

- [1] https://www.cs.swarthmore.edu/~bryce/cs63/s16/slides/3-21_value_iteration.pdf
- [2] <https://github.com/JaeDukSeo/reinforcement-learning-an-introduction>

• • •

WRITTEN BY

Jeremy Zhang

[See all from Jeremy Zhang](#)

Topics:

Machine Learning

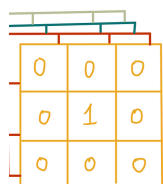
Python

Reinforcement Learning

Share this article:



Related Articles



ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in



ARTIFICIAL INTELLIGENCE

How to Forecast Hierarchical Time Series

Reinforcement Learning in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao

August 20, 2024 6 min read

Reinforcement Learning Time Series

A beginner's guide to forecast reconciliation

Dr. Robert Kübler

August 20, 2024 13 min read

DATA SCIENCE

Hands-on Time Series Anomaly Detection using Autoencoders, with Python

Here's how to use Autoencoders to detect signals with anomalies in a few lines of...

Piero Paialunga

August 21, 2024 12 min read

MACHINE LEARNING

3 AI Use Cases (That Are Not a Chatbot)

Feature engineering, structuring unstructured data, and lead scoring

Shaw Talebi

August 21, 2024 7 min read

DATA SCIENCE

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao

February 3, 2023 6 min read

DATA SCIENCE

Must-Know in Statistics: The Bivariate Normal Projection Explained

Derivation and practical examples of this powerful concept

Luigi Battistoni

August 14, 2024 7 min read

DATA SCIENCE

Our Columns

Columns on TDS are carefully curated collections of posts on a particular idea or category...

TDS Editors

November 14, 2020 4 min read

DATA SCIENCE

Optimizing Marketing Campaigns with Budgeted Multi-Armed Bandits

With demos, our new solution, and a video

Vadim Arzamasov

August 16, 2024 10 min read

DATA SCIENCE

Back to Basics, Part Tres: Logistic Regression

An illustrated guide to everything you need to know about Logistic Regression

Shreya Rao

March 2, 2023 8 min read



Your home for data science and AI. The world's leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2025

[ABOUT](#) · [PRIVACY POLICY](#) · [TERMS OF USE](#)

Towards Data Science is now independent!

[COOKIES SETTINGS](#)

Sign up to our newsletter

Email address*

First name*

Last name*

Job title*

Job level*

Company name*



I consent to receive newsletters and other communications from Towards Data Science publications.*

Subscribe Now